

SUBMITTED BY-

MT2024121 Priyansh Rai

MT2024141 Shashank Vyas

Distributed Key-Value Store

1. Abstract

This project presents the design and implementation of a fault-tolerant, distributed key-value store built using Python, FastAPI, Docker, and Kubernetes. The system demonstrates fundamental distributed systems concepts such as consistent hashing, replication, quorum-based writes, heartbeat monitoring, failure detection, and automatic rebalancing. The system is deployed using Minikube Kubernetes cluster and simulates real-world distributed storage engines like Cassandra, DynamoDB, and etcd in a simplified but functional manner. The project aims to provide clear understanding of how distributed databases maintain availability and data correctness even under node failures.

2. Introduction

Modern distributed systems support large-scale applications that require high availability, low latency, and fault tolerance. Traditional single-node architectures cannot handle such requirements because they become bottlenecks and single points of failure. Distributed storage systems such as DynamoDB, Cassandra, Chubby, Bigtable, and etcd use replication, consistent hashing, and membership monitoring to distribute data across multiple nodes while ensuring durability and scalability.

This project implements a smaller, educational version of such a system. It contains one dedicated **controller service** that manages cluster state, and four **worker services** that store and replicate key-value pairs. The communication between services is performed using REST APIs through FastAPI. Kubernetes is used to orchestrate the services, provide container management, pod restarts, and stable networking. The goal of this project is to demonstrate how distributed storage engines maintain consistency and availability during failures.

3. System Overview

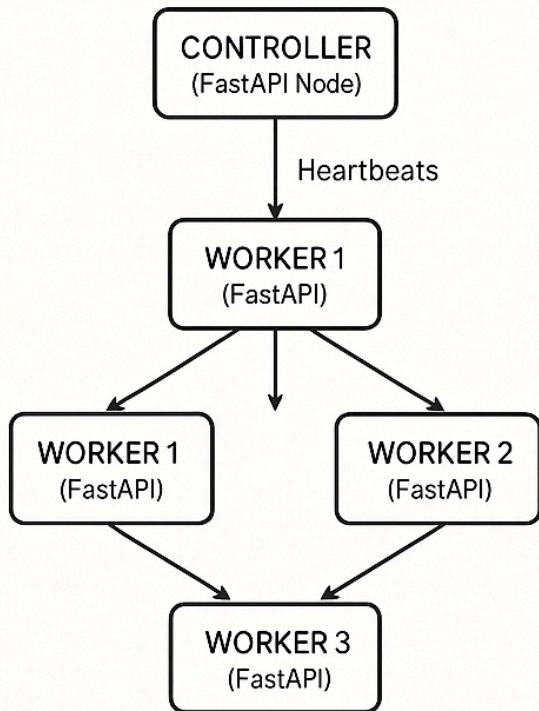
The system consists of:

- **Controller Node**
Responsible for membership management, heartbeat tracking, consistent hashing, and notifying workers when a node fails.
- **Worker Nodes (4 nodes)**
Each worker stores a partition of the key-value data. Workers replicate each key to two additional workers to maintain a replication factor of three. Workers handle PUT and GET operations and perform rebalancing after failures.
- **Kubernetes Deployment**
Each service runs as a Kubernetes Deployment with an associated Service object for internal DNS-based communication. Minikube is used to run Kubernetes locally.

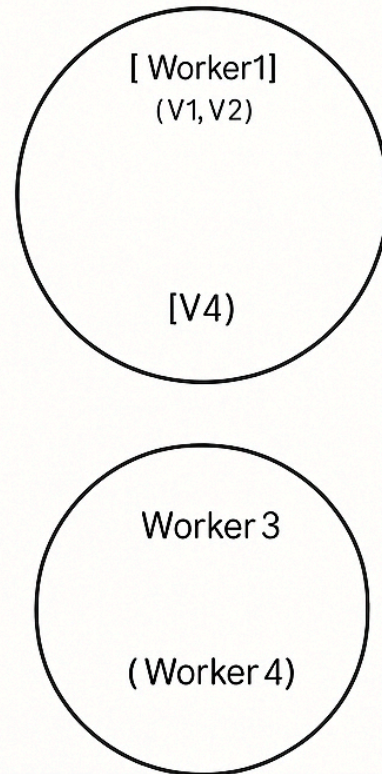
The communication flow in the system is simple:

Controller manages node status and mapping; Workers store data and replicate it according to the hash ring.

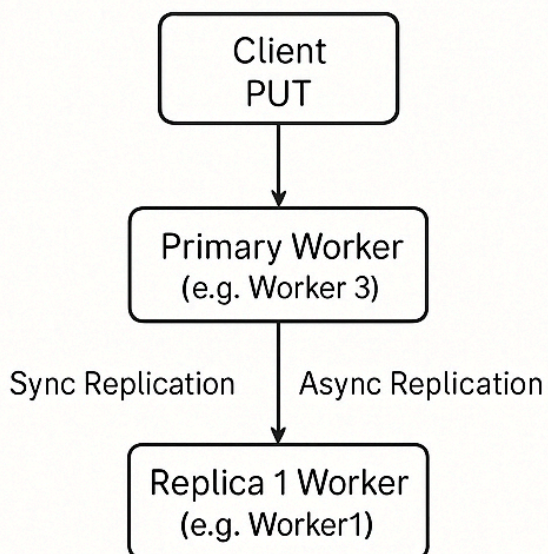
System Architecture



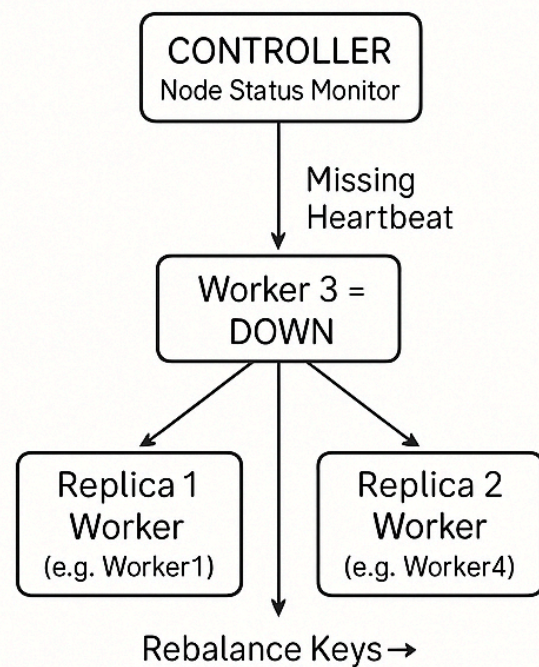
Consistent Hashing Ring



Replication Flow (Write Operation)



Failure Detection + Rebalancing



4. Consistent Hashing and Data Distribution

Consistent hashing is the backbone of this distributed key-value store. It allows keys to be distributed evenly across workers while minimizing data movement when nodes join or leave the system. The controller constructs a **hash ring** using SHA-256. Each worker is assigned multiple **virtual nodes** on the ring to improve load balancing.

When a key is inserted:

1. The key is hashed using SHA-256
2. The system finds the next clockwise virtual node on the ring
3. The corresponding worker becomes the **primary replica**
4. The next two nodes clockwise become **replica 1** and **replica 2**

This ensures a replication factor of **3**, reducing chances of data loss during node failure.

The biggest advantage of consistent hashing is that only keys belonging to the failed or new node's region need to be moved. This minimizes rebalancing workload.

5. Replication Strategy and Write Quorum

Every write operation follows a primary-replica model:

1. The client sends a PUT request to the **primary worker**
2. The primary writes the key locally
3. The primary performs a **synchronous replication** to replica1
4. The primary triggers **asynchronous replication** to replica2

This gives the following guarantees:

- **Replication Factor = 3**
- **Write Quorum = 2** (primary + sync replica)
- **Fault Tolerance:** System can still accept writes even if one node is down
- **Eventual Consistency:** All replicas converge but slight delays may exist for the async replica

This design mirrors real distributed databases such as DynamoDB and Cassandra.

6. Heartbeat Mechanism and Failure Detection

Each worker periodically sends a **heartbeat** to the controller through `/heartbeat` endpoint. The controller monitors timestamps of these heartbeats in a loop. If a worker does not send a heartbeat for a configured timeout period, it is declared **DOWN**.

When a worker is marked DOWN:

1. Controller updates its node registry
2. Controller triggers a `/node_down` event to all healthy workers
3. Workers begin rebalancing their keys according to the updated hash ring

This ensures fast and accurate failure detection, similar to systems like etcd and Chubby.

7. Rebalancing and Recovery Process

When a worker node fails, the consistency of replication must be restored. The rebalancing process is triggered by the controller and executed by each worker as follows:

1. Worker receives `/node_down` notification
2. Worker iterates through its local key store

3. For each key, the worker requests the updated mapping `/mapping` from the controller
4. If the worker should now hold the key (as per consistent hashing), it ensures the key is stored
5. If other workers need the key, the current worker replicates it to them

In this distributed approach, workers cooperatively restore replication factor rather than relying on a single recovery node. This decentralized repair reduces bottlenecks and keeps the system available.

8. Kubernetes Deployment

Kubernetes provides container orchestration, fault tolerance, and networking. Each node (controller or worker) runs inside a separate Kubernetes Deployment. Service objects ensure stable DNS access to each node (e.g., `worker1.kv-system.svc.cluster.local`). Minikube is used for local Kubernetes simulation.

Advantages of Kubernetes in this project:

- Automatic restart on failure
- Scaling workers up/down effortlessly
- Namespace-based isolation
- Simple failure simulation through pod deletion
- Stable inter-service communication with Services
- Port-forwarding for local testing

This deployment architecture closely resembles real-world microservice clusters.

9. Experimental Evaluation

Test 1 — Consistent Hashing

Keys were mapped to workers using the hash ring. Virtual nodes ensured uniform load distribution across all workers.

Test 2 — Replication Verification

After writing a key to the primary, both replica1 and replica2 successfully received copies. Quorum-based consistency was maintained.

Test 3 — Failure Detection

Scaling a worker to zero replicas (`kubectl scale`) immediately triggered the controller's failure detection logic. The worker was marked DOWN within the timeout windows.

Test 4 — Rebalancing

Workers successfully performed rebalancing after a failure. Replicas were restored on remaining nodes, maintaining replication factor = 3.

Test 5 — Kubernetes Resiliency

Deleting any pod forces Kubernetes to respawn it. The system remained functional through pod restarts and node failures.

10. Conclusion

This project presents a functional distributed key-value storage system built from first principles. The system integrates consistent hashing, replication, quorum-based writes, failure detection, and rebalancing. Using Python and Kubernetes, it simulates many aspects of real distributed storage engines in a lightweight and understandable manner.

The project demonstrates:

- How distributed systems maintain data availability
- How failures are detected and recovered
- How hashing and replication improve scalability
- How Kubernetes acts as a robust orchestration platform

Overall, this is a practical demonstration of real-world distributed system design.

11. Future Work

Possible extensions include:

- Implementing tunable consistency levels (e.g., QUORUM, ALL, ONE)
- Adding persistent storage (SQLite, RocksDB) instead of in-memory data
- Using gossip-based membership protocols
- Integrating Prometheus/Grafana for monitoring
- Supporting multi-region deployments
- Implementing vector clocks or CRDTs for conflict resolution
- Adding batching or hinted handoff for improved recovery

These features would further align the system with modern production-ready distributed databases.
